

Lab 4 Exam

这道题主要考察的是如何添加新的 syscall，这也是每年必出的经典考点。接下来，我们根据“实现参考”部分的思路走一遍整个流程：

(1) 在 `include/syscall.h` 中新增一个枚举值 `SYS_get_ppid`，注意，需要将它放在 `MAX_SYSNO` 之前！

这一步没什么好说的，我们只需要将 `SYS_get_ppid` 放在 `MAX_SYSNO` 之前的任意位置即可：

```
// include/syscall.h
enum {
    SYS_putchar,
    SYS_print_cons,
    SYS_getenv_id,
    .....
    SYS_get_ppid,
    MAX_SYSNO,
};
```

(2) 在 `kern/syscall_all.c` 中增加一个系统调用实现 `int sys_get_ppid(void) { ... }`，并将其添加到 `syscall_table` 中。注意，实现函数的位置应该在系统调用表之前。

在这一步，我们首先填写 `syscall_table`，其中数组下标为我们刚才填入的枚举值 `SYS_get_ppid`，数组值为该文件中实现 syscall 的函数名 `sys_get_ppid`。

接下来，我们来完成 `sys_get_ppid()` 函数。`kern/syscall_all.c` 中 `sys` 开头的函数负责 syscall 的最终实现，所以没有固定的格式，需要我们根据题目要求自行编写。题目中要求我们返回当前进程的父进程的 `env_id`，非常幸运的是，这个内容正好就写在了当前进程的 PCB 中，于是我们只需要调用 `curenv->env_parent_id`，即可得到我们想要的结果。

顺带一提，题目中提到了如果当前进程没有父进程，需要返回 0，然而实际上在我们的操作系统中，当一个进程没有父进程时，它的 `env_parent_id` 字段本身就是 0，所以不需要特判。不过如果题目中规定没有父进程的时候需要返回一些其它的值，我们也需要会使用 `env_parent_id` 字段是否为 0 进行判断。（不用担心父进程的 `env_id` 刚好等于 0 的情况，如果你研究过代码，就会发现一个进程的 `env_id` 不可能是 0！）

```
// kern/syscall_all.c
int sys_get_ppid(void) {
    return curenv->env_parent_id;
}

void *syscall_table[MAX_SYSN0] = {
    [SYS_putchar] = sys_putchar,
    [SYS_print_cons] = sys_print_cons,
    .....,
    [SYS_get_ppid] = sys_get_ppid,
};
```

(3) 在 `user/include/lib.h` 中增加一个用户态的系统调用声明：`int syscall_get_parent_envid(void);`。

这一步也是简单到爆，直接将 `int syscall_get_parent_envid(void);` 写进文件中就可以了，这里就不贴代码了。

(4) 在 `user/lib/syscall_lib.c` 中增加它的具体实现，你可以通过 `msyscall` 发起系统调用。

我们可以仿照 `user/lib/syscall_lib.c` 中的其它函数来实现 `syscall_get_parent_envid()` 函数。这个函数也是固定套路了，我们只需要写上一行 `return` 语句，返回 `msyscall()` 函数的执行结果。其中，`msyscall()` 的第一个参数为之前填入的枚举值，接下来的参数为 `kern/syscall_all.c` 中 `sys` 开头的函数所需要的参数。这里 `sys_get_ppid()` 函数不需要参数，所以我们只需要向 `msyscall()` 函数传入一个枚举值 `SYS_get_ppid` 即可：

```
// user/lib/syscall_lib.c
int syscall_get_parent_envid(void) {
    return msyscall(SYS_get_ppid);
}
```

这样，我们就完成了 exam 的第一部分，练习了如何添加一个新的 syscall！接下来我们来挑战一下非常简单的第二部分：

在这一部分，我们需要实现一个新的页表项标志位 `PTE_PROTECT`。同时，在 `fork` 子进程，将子进程的虚拟地址与物理页面进行映射时，我们不应该映射具有 `PTE_PROTECT` 项的物理页面。

听起来还是比较复杂的，不过我们只要跟着“实现参考”中的思路走，就会发现这其实是一个非常简单的问题！首先，我们需要在 `include/mmu.h` 中添加一行定义 `#define PTE_PROTECT 0x0004`。这里直接添加即可，我们来看下一步。

接下来的提示可以说是差点把答案写脸上了，我们需要在 `user/lib/fork.c` 中 `duppage()` 函数的 Step 1 和 Step 2 之间检查当前页面是否具有 `PTE_PROTECT` 标志位，如果具有则直接结束函数。于是我们直接使用 Step 1 中计算出的 `perm`，与 `PTE_PROTECT` 进行按位与，检查该结果是否为 0 即可：

```
// user/lib/fork.c
static void duppage(u_int envid, u_int vpn) {
    int r;
    u_int addr;
    u_int perm;

    /* Step 1: Get the permission of the page. */
    /* Hint: Use 'vpt' to find the page table entry. */
    /* Exercise 4.10: Your code here. (1/2) */

    addr = vpn << PGSHIFT;
    perm = vpt[vpn] & 0xfff;

    if (perm & PTE_PROTECT) {
        return;
    }

    /* Step 2: If the page is writable, and not shared
     * then map it as copy-on-write, both in the parent
     * Hint: The page should be first mapped to the chi
     */
    /* Exercise 4.10: Your code here. (2/2) */

    .....
}
```

不过说实话，即使是这么简单的第二部分，我还是被硬控了好一阵，因为我一开始把判断条件写成了这样：if (perm & PTE_PROTECT != 0)。发现哪里有问题了吗？答案是在 C 语言中，!= 的运算优先级要高于 &，于是整个就全错了！大家千万补药在考场上犯这种错误口牙！

Lab 4 Extra

这道题目看起来还是非常仁慈的，已经给出了完整的 syscall 的实现过程，我们只需要实现内核态中的 4 个函数即可。

首先我们来快速完成一下 user/lib/syscall_lib.c 中的函数封装，我们只需要返回正确的 msyscall() 函数调用即可。仿照该文件中的其它函数，我们在 msyscall() 函数的第一个参数中，填入每个 syscall 在 kern/syscall_all.c 的 syscall_table 中对应的下标；接下来，我们再向 msyscall() 函数传递一些参数，作为 kern/syscall_all.c 中 sys 开头的函数的参数：

```
// user/lib/syscall_lib.c
int syscall_shm_new(u_int npage) {
    // Lab4-Extra: Your code here. (1/8)
```

```

    return msyscall(SYS_shm_new, npage);
}

int syscall_shm_bind(int key, u_int va, u_int perm) {
    // Lab4-Extra: Your code here. (2/8)
    return msyscall(SYS_shm_bind, key, va, perm);
}

int syscall_shm_unbind(int key, u_int va) {
    // Lab4-Extra: Your code here. (3/8)
    return msyscall(SYS_shm_unbind, key, va);
}

int syscall_shm_free(int key) {
    // Lab4-Extra: Your code here. (4/8)
    return msyscall(SYS_shm_free, key);
}

```

接下来就是实现 4 个内核态函数的部分了，这里我们千万不要被题目中的 Shm 结构体给唬住了，我们只要观察 Shm 结构体的属性，就会发现 Shm 结构体只不过是装着几个物理页面的容器，我们只需要对这些物理页面进行申请页面、添加映射、解除映射、释放页面这四个操作即可。这些操作在 kern/pmap.c 中都给出了相应的函数，就看我们能否调用正确了！

话虽如此，实际上要想做对这道题还是比较有难度的，因为它和一般的页面分配流程不太一样，你发现哪里有问题了吗？关键就在于释放页面这一步。在这道题中，我们需要通过调用 sys_shm_free() 函数实现手动释放页面，这和 kern/pmap.c 中原有函数的设计是冲突的。

在原有的设计中，我们在使用 page_insert() 函数添加映射时会使物理页面的 pp_ref 字段增加 1，使用 page_remove() 函数解除映射时会通过调用 page_decref() 函数使物理页面的 pp_ref 字段减少 1。在 page_decref() 函数中，当 pp_ref 减少到 0 时，会自动调用 page_free() 函数将页面释放掉，这样就实现了页面的自动释放。

而在本道题中，对于 Shm 结构体中的物理页面，即使所有的映射都被解除，我们也不应该释放这些页面，而是应该保证在此之后依然能够对相同的页面进行映射。那么我们该如何在不改变原有函数的情况下实现这样的操作呢？实际上题目中已经给出了提示：你可以通过在创建共享内存时，增加页面的 pp_ref 记录，并在销毁共享内存时使用 page_decref() 函数释放页面。

也就是说，我们在使用 sys_shm_new() 函数申请页面的时候，就将每个页面的 pp_ref++，这样即使所有的映射都被解除，页面的 pp_ref 依然有 1，不会触发自动释放；而我们在 sys_shm_free() 函数中，只需要调用一次 page_decref() 函数，即可将页面的 pp_ref 减少至 0，并且自动释放页面。非常巧妙的写法！

解决了最关键的问题之后，我们就不难完成这道题了。首先是 sys_shm_new() 函数。我们首先遍历所有的 Shm 结构体，直到找到一个 open 字段为 0 的结构体；如果没有找到，则返回 -E_SHM_INVALID。接下来，我们使用 page_alloc() 函数申请 npage 个页面，将其写入当前 Shm 结构体的 pages 字段中，并将页面的 pp_ref++。如果页面申请失败，即 page_alloc() 函数的返回值小于 0，那么我们就需要将之前所有已申请的页面释放掉，此时只需要对之前的每个页面调用 decref() 函数，并将其从 Shm 结构体的 pages 字段中删除即可。最后，我们维护当前 Shm 结构体的 npage 和 open 字段，这样就完成了 sys_shm_new() 函数：

```

// kern/syscall_all.c
int sys_shm_new(u_int npage) {
    if (npage == 0 || npage > N_SHM_PAGE) {
        return -E_SHM_INVALID;
    }

    // Lab4-Extra: Your code here. (5/8)
    int whichShm = -1;
    for (int i = 0; i < N_SHM; i++) {
        if (shm_pool[i].open == 0) {
            whichShm = i;
            break;
        }
    }
    if (whichShm == -1) {
        return -E_SHM_INVALID;
    }

    for (int i = 0; i < npage; i++) {
        struct Page *p;
        int r = page_alloc(&p);
        if (r != 0) {
            for (int j = 0; j < i; j++) {
                struct Page *pp = shm_pool[whichShm].pages[j];
                page_decref(pp);
                shm_pool[whichShm].pages[j] = NULL;
            }
            return r;
        }
        p->pp_ref++;
        shm_pool[whichShm].pages[i] = p;
    }

    shm_pool[whichShm].npage = npage;
    shm_pool[whichShm].open = 1;

    return whichShm;
}

```

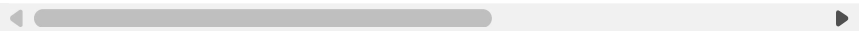
接下来是 `sys_shm_bind()` 函数。我们首先检查 `shm_pool[key]` 的 `open` 字段是否为 1，如果为 0 则返回 `-E_SHM_NOT_OPEN`。接下来，我们使用 `page_insert()` 函数，将从 `va` 开始连续 `shm_pool[key].npage` 个页面的虚拟地址分别映射到 `shm_pool[key].pages` 中的前 `shm_pool[key].npage` 个页面上：

```
// kern/syscall_all.c
int sys_shm_bind(int key, u_int va, u_int perm) {
    if (key < 0 || key >= N_SHM) {
        return -E_SHM_INVALID;
    }

    // Lab4-Extra: Your code here. (6/8)
    if (shm_pool[key].open == 0) {
        return -E_SHM_NOT_OPEN;
    }

    for (int i = 0; i < shm_pool[key].npage; i++) {
        u_int nowva = va + i * PAGE_SIZE;
        page_insert(curenv->env_pgdir, curenv->env_asic
    }

    return 0;
}
```



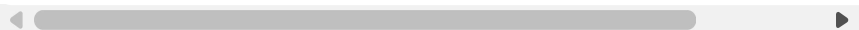
sys_shm_unbind() 函数也是同样的操作，这一次我们使用 page_remove() 函数：

```
// kern/syscall_all.c
int sys_shm_unbind(int key, u_int va) {
    if (key < 0 || key >= N_SHM) {
        return -E_SHM_INVALID;
    }

    // Lab4-Extra: Your code here. (7/8)
    if (shm_pool[key].open == 0) {
        return -E_SHM_NOT_OPEN;
    }

    for (int i = 0; i < shm_pool[key].npage; i++) {
        u_int nowva = va + i * PAGE_SIZE;
        page_remove(curenv->env_pgdir, curenv->env_asic
    }

    return 0;
}
```



最后是 `sys_shm_free()` 函数，我们同样需要检查 `shm_pool[key]` 的 `open` 字段，并且需要将 `open` 字段置为 0，这样才能保证当前的 `Shm` 结构体能够被重新分配。接下来，我们对 `shm_pool[key].pages` 中的 `shm_pool[key].npage` 个页面，使用 `page_decref()` 函数进行释放，并将这些页面从 `shm_pool[key].pages` 中移除：

```
// kern/syscall_all.c
int sys_shm_free(int key) {
    if (key < 0 || key >= N_SHM) {
        return -E_SHM_INVALID;
    }

    // Lab4-Extra: Your code here. (8/8)
    if (shm_pool[key].open == 0) {
        return -E_SHM_NOT_OPEN;
    }

    shm_pool[key].open = 0;
    for (int i = 0; i < shm_pool[key].npage; i++) {
        struct Page *p = shm_pool[key].pages[i];
        // printk("%d\n", pp->pp_ref);
        page_decref(p);
        shm_pool[key].pages[i] = NULL;
    }

    return 0;
}
```

这样我们就完成了整道问题！实际上，如果能够想明白在申请页面和释放页面是分别进行 `pp_ref++` 和 `pp_ref--`，那这道题就没有什么难度了。然而在考场上我硬是研究了好久，一是那个时候确实对代码不够熟悉，二是没有仔细看题目中的提示。实际上题目中已经明晃晃地给出了解决方案，所以千万别看题干长就不读了，救赎之道，就在其中！